ИТМО

Алгоритмы и структуры данных

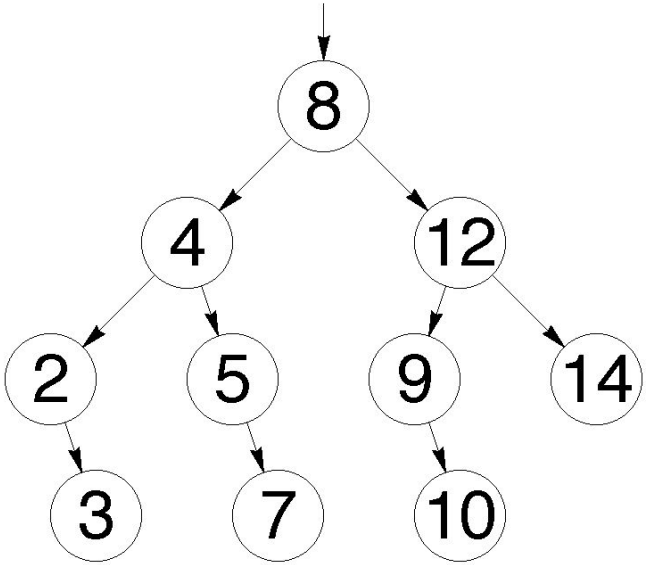
Лекция 6

Бинарные деревья поиска и красно-черные деревья

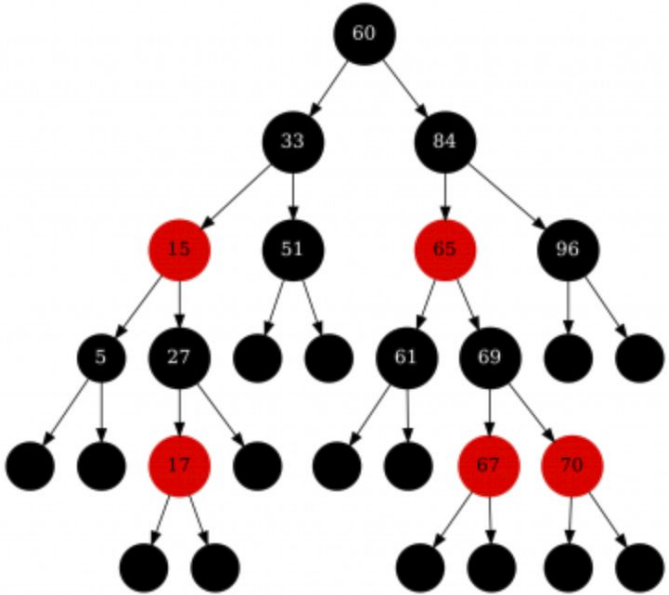
Бинарные деревья поиска и красно-черные деревья

ИТМО

Бинарные деревья



Красно-черные деревья



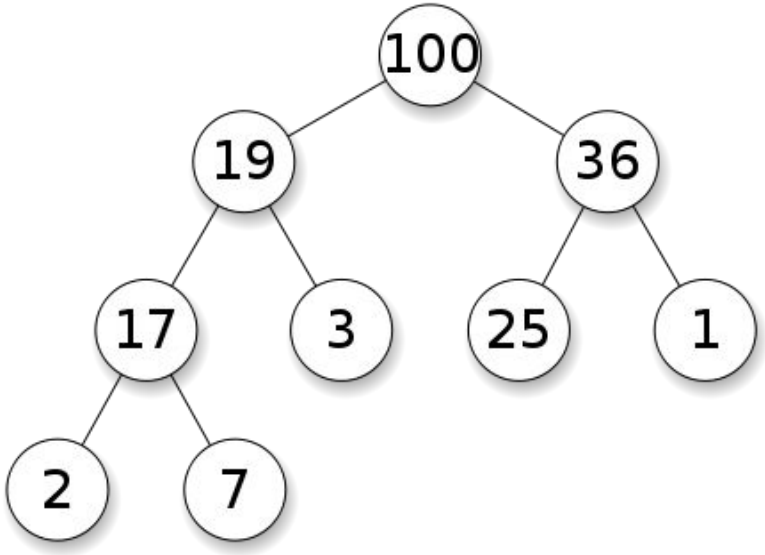
Применение:

- Индексация в базах данных
- Синтаксический парсинг
- Decision Tree (Дерево решений)

ЗАЧЕМ???

Утверждается, что в полном бинарном дереве

Полное* бинарное дерево



*все уровни, кроме, возможно, последнего, заполнены полностью, и все узлы последнего уровня располагаются максимально слева.

Операции:

- Поиск
- Минимум
- Максимум
- Предыдущий элемент
- Следующий элемент
- Вставка
- Удаление

выполняются за **O(lg(N))** в наихудшем случае

Допустим есть бд и индекс

ИТМО

id	P	cost
1	18	100
2	38	50
5	28	20
10	22	150
11	44	99

cost	id
20	5
50	2
99	11
100	1
150	10

И есть бинарный поиск

O(log(N))

В чем может быть проблема?

id == 5

cost == 20

Вставка новых значений - бессердечная ты ***

ИТМО

id	P	cost
1	18	100
2	38	50
5	28	20
10	22	150
11	44	99
12	11	51

cost	id
20	5
50	2
51	12
99	11
100	1
150	10

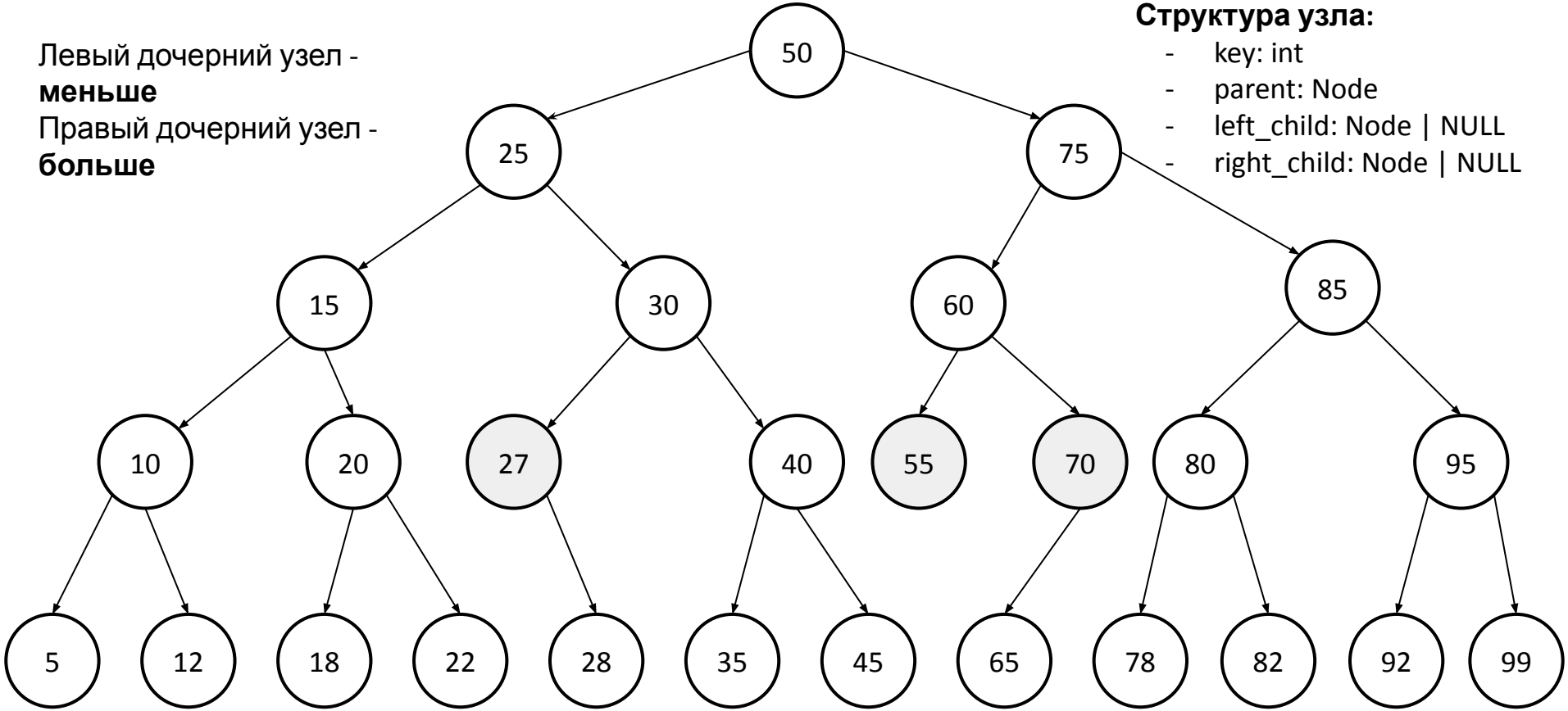
Вставка в список - <u>это грустно</u>

Свойства бинарного дерева

Левый дочерний узел - **меньше**
Правый дочерний узел - **больше**

Структура узла:

- key: int
- parent: Node
- left_child: Node | NULL
- right_child: Node | NULL



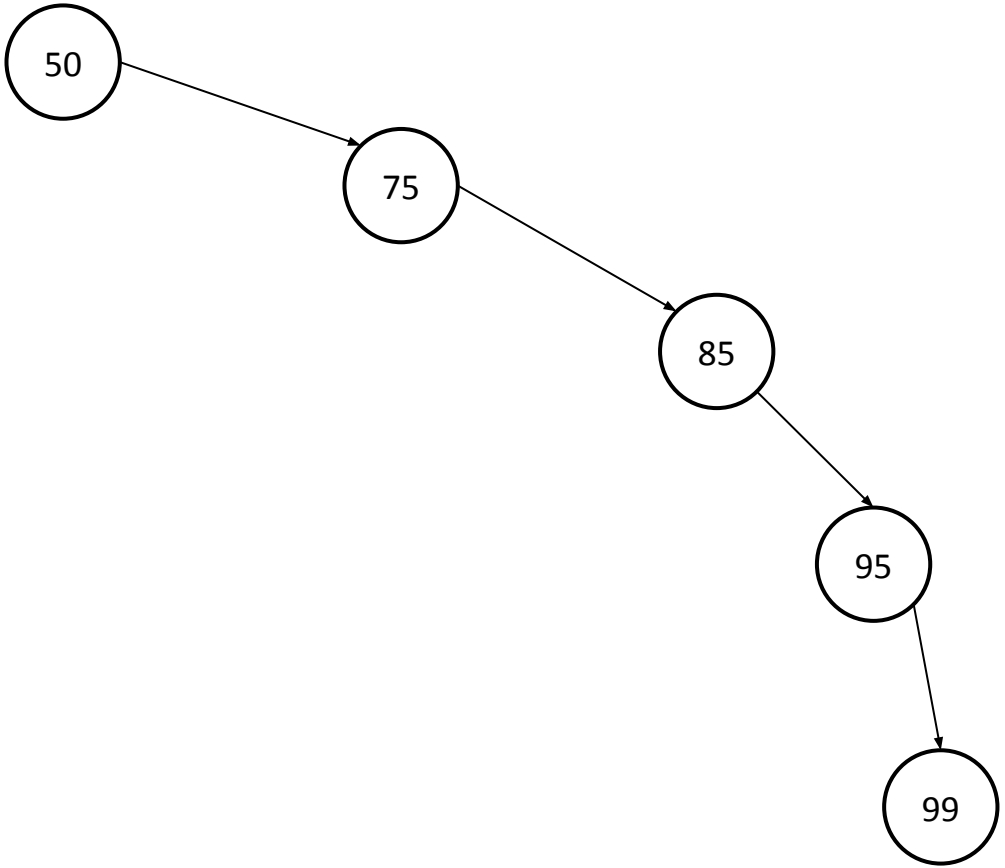
А это бинарное дерево?

Левый дочерний узел - **меньше**
Правый дочерний узел - **больше**

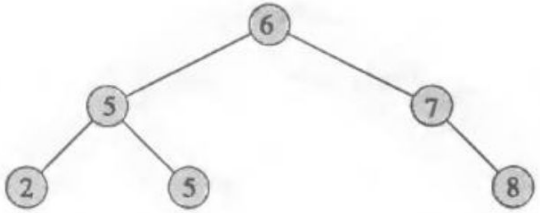
N - размер дерева
K - высота дерева
В данном случае N == K

Какая будет сложность?

- Поиск
- Минимум
- Максимум
- Предыдущий элемент
- Следующий элемент
- Вставка
- Удаление



Есть 3 обхода бинарного дерева



ИТМО

Обход в прямом порядке

```
void preorder_tree_walk (Node* x) {
    // Обход в прямом порядке.
    // Сначала выводится корень,
    // а затем - значения левого
    // и правого поддеревьев
    if (x != nullptr) {
        std::cout << x->key << " ";
        preorder_tree_walk (x->left);
        preorder_tree_walk (x->right);
    }
}
```

6 5 2 5 7 8

Центрированный (симметричный)

```
void inorder_tree_walk(Node* x) {
    // Центрированный (симметричный)
    // Ключ в корне выводится между
    // значениями ключей левого и
    // правого поддеревьев.
    if (x != nullptr) {
        inorder_tree_walk(x->left);
        std::cout << x->key << " ";
        inorder_tree_walk(x->right);
    }
}
```

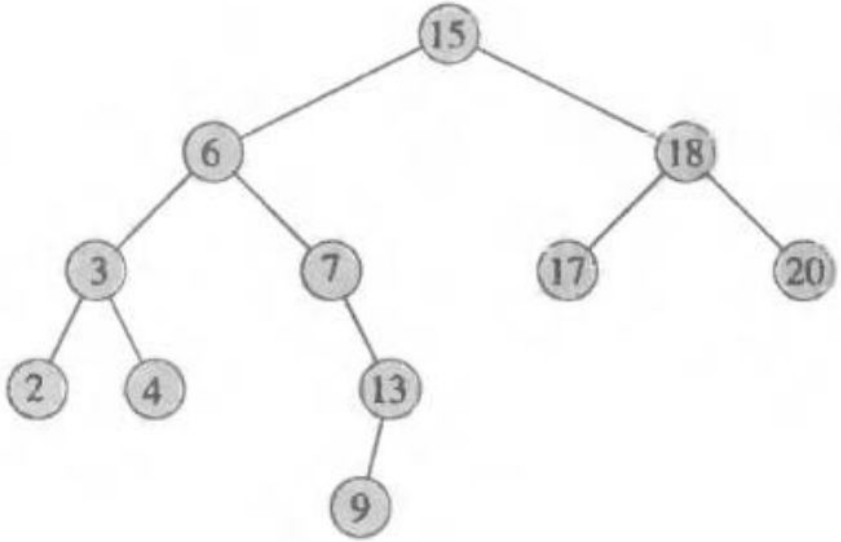
2 5 5 6 7 8

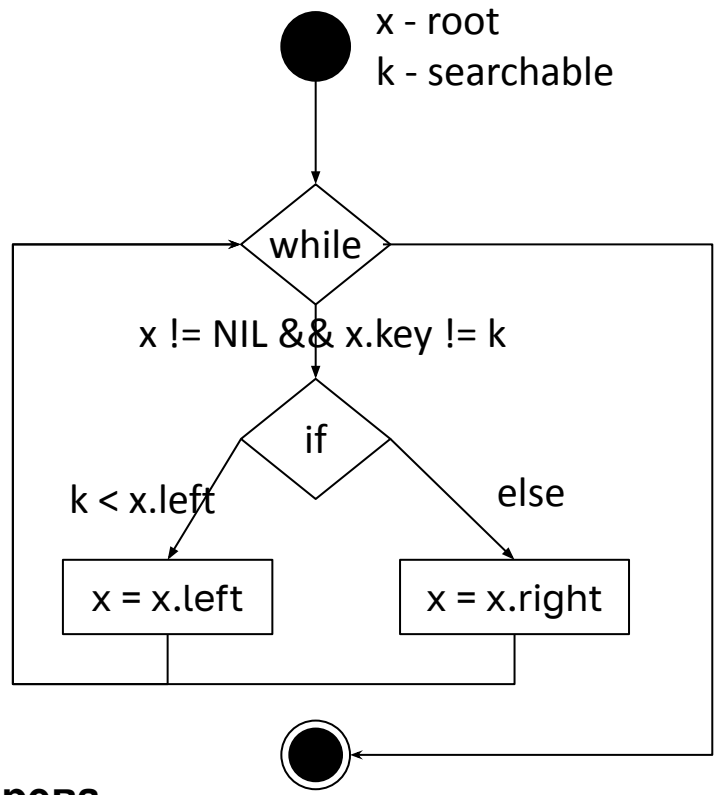
Обход в обратном порядке

```
void postorder_tree_walk(Node* x) {
    // Обход в обратном порядке.
    // Первыми выводятся значения
    // левого и правого поддеревьев,
    // а уже затем - корня.
    if (x != nullptr) {
        postorder_tree_walk(x->left);
        postorder_tree_walk(x->right);
        std::cout << x->key << " ";
    }
}
```

2 5 5 8 7 6

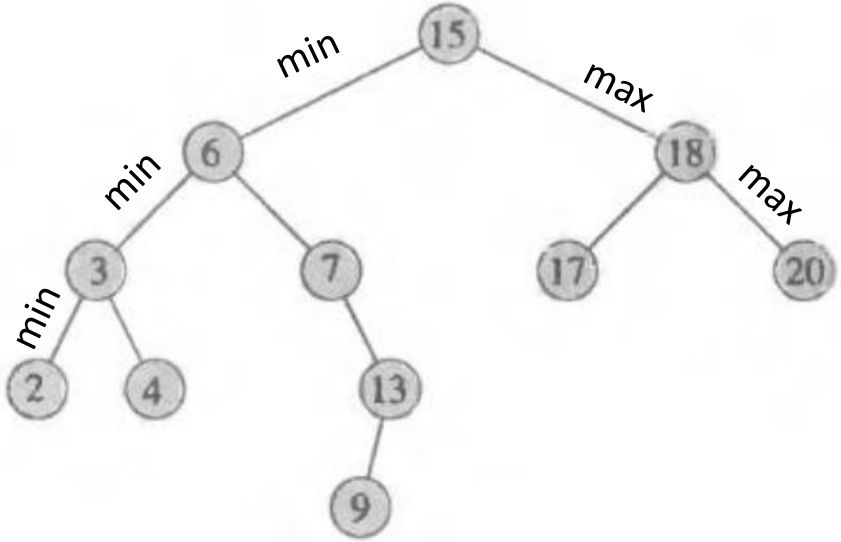
Поиск в бинарном дереве





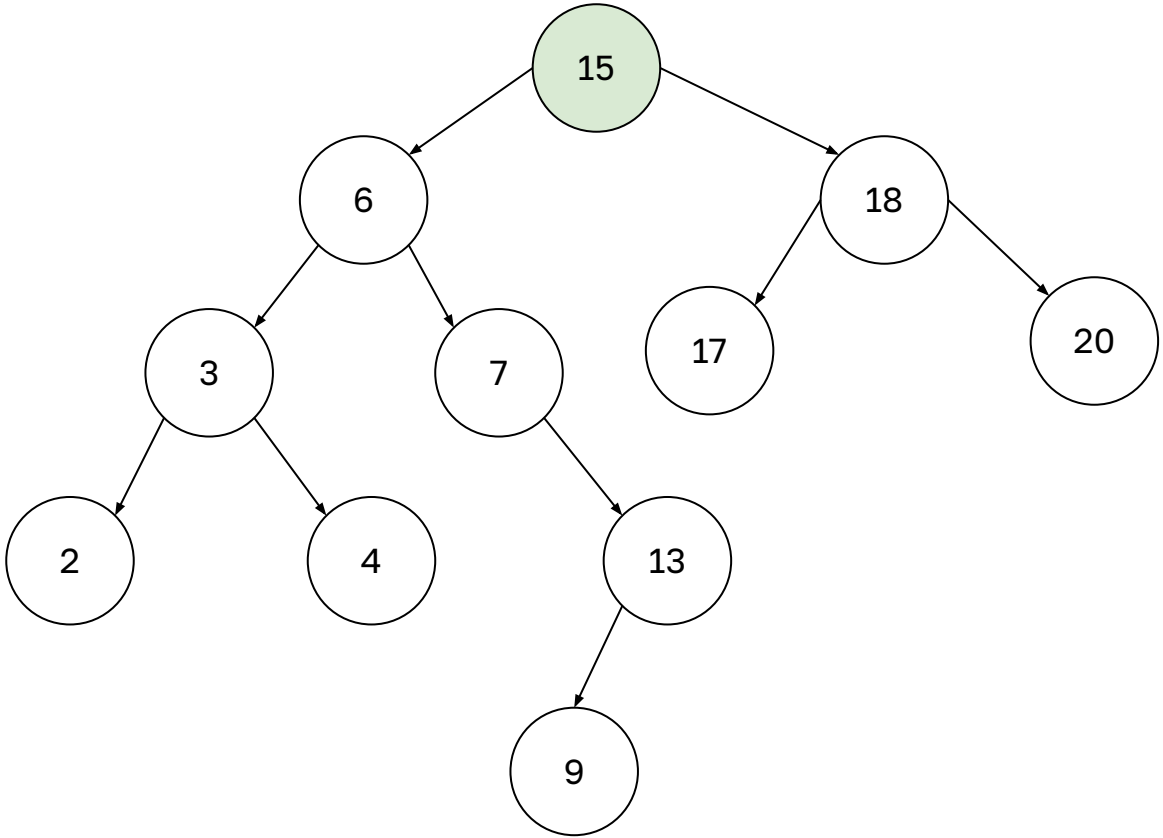
O(h), h - высота дерева

min и max в бинарном дереве



O(h), h - высота дерева

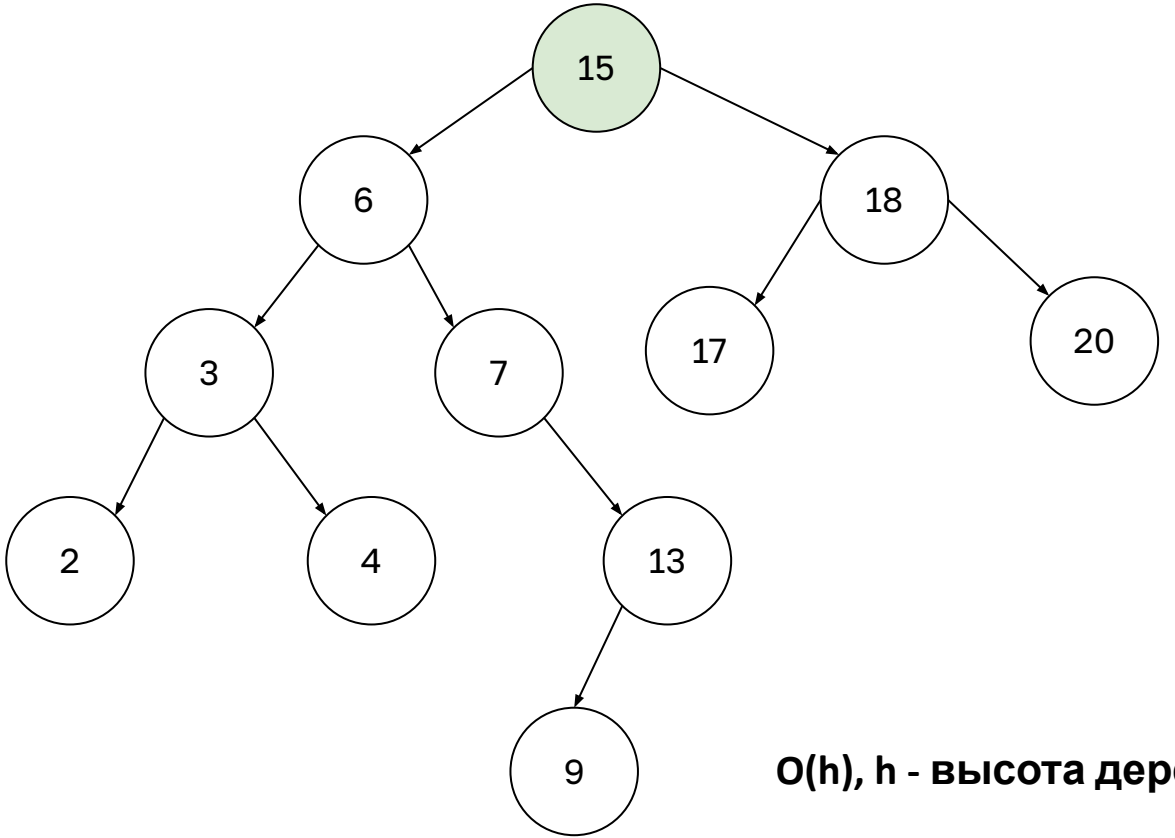
Предшествующий и последующий элементы

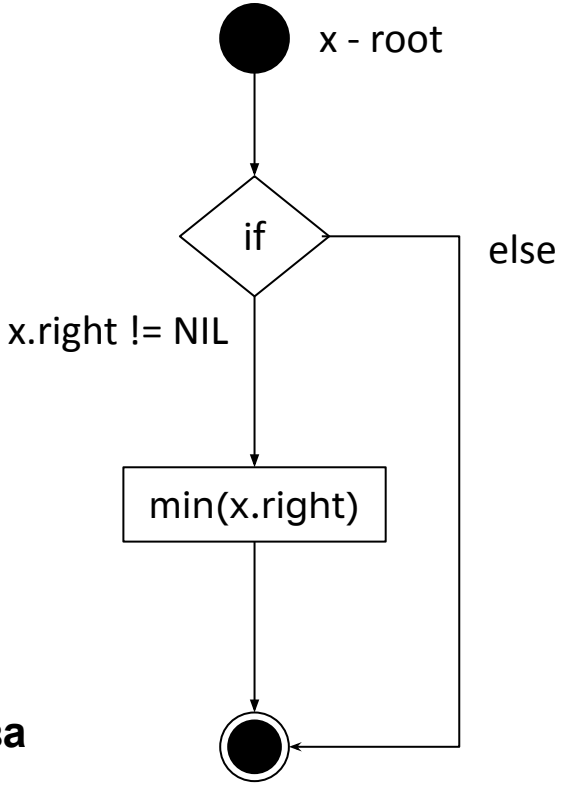


Давайте сами…

O(h), h - высота дерева

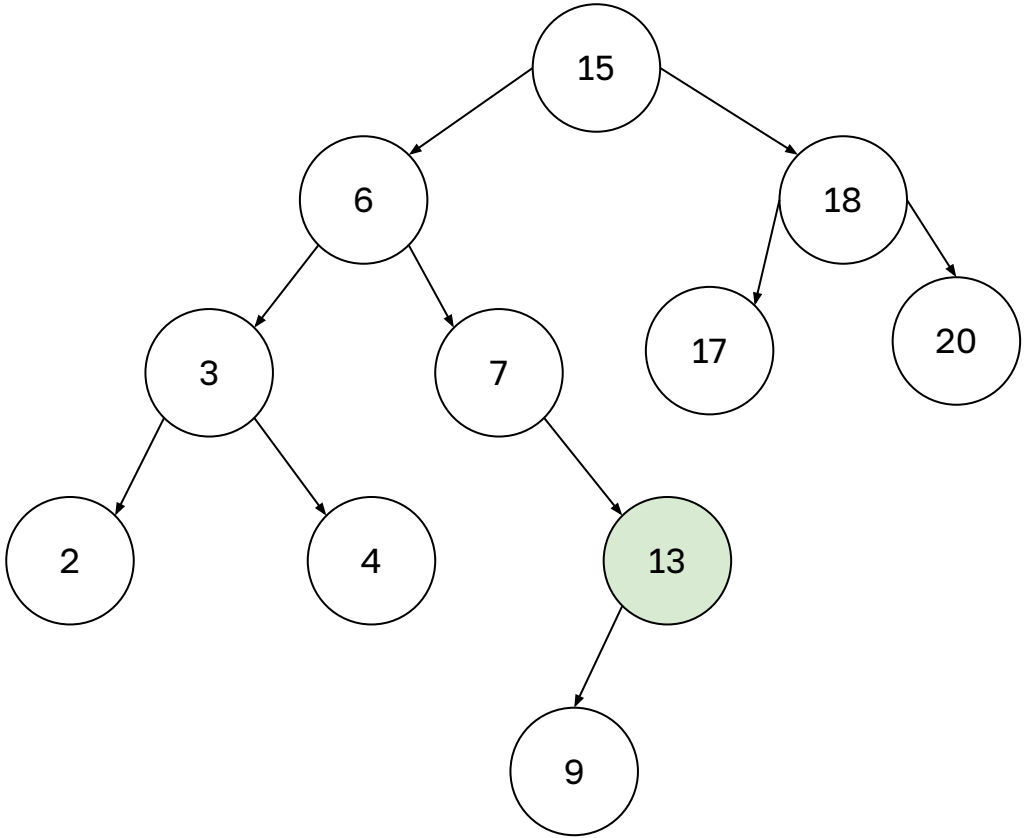
Предшествующий и последующий элементы





O(h), h - высота дерева

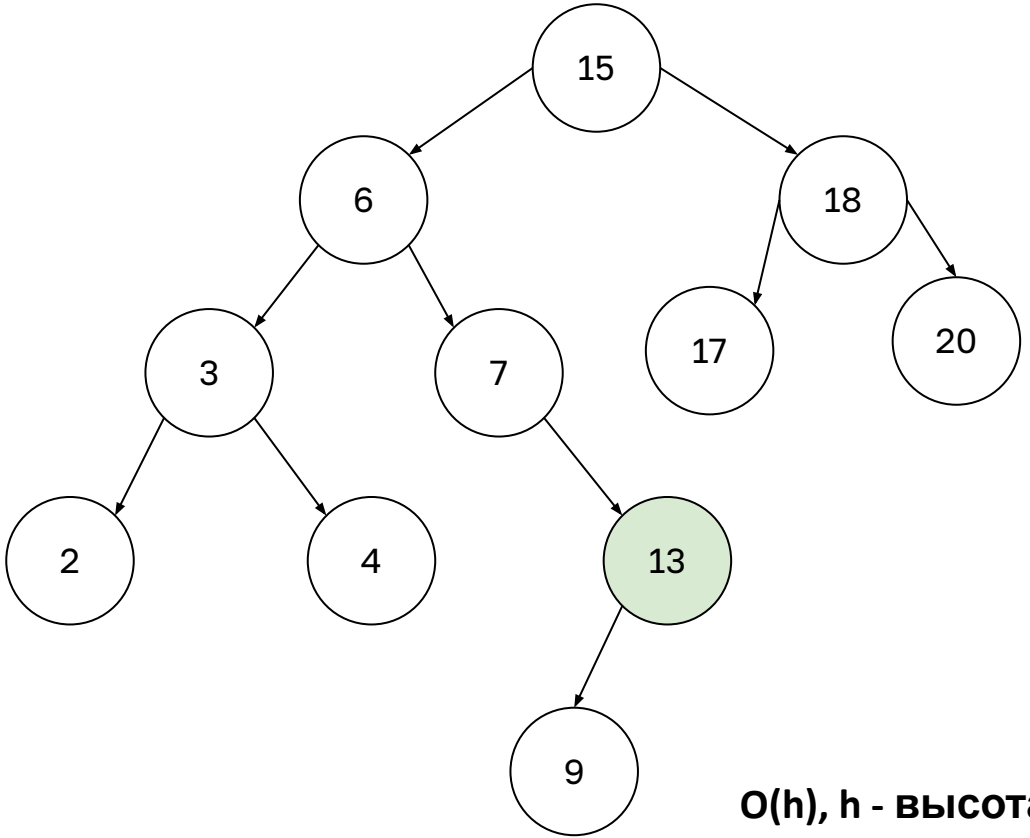
Предшествующий и последующий элементы

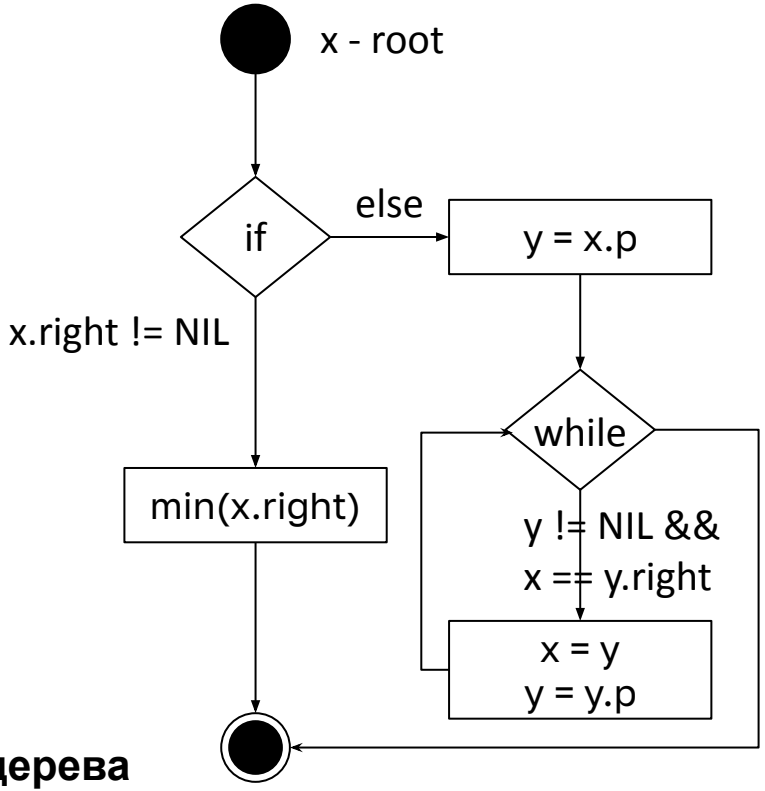


Давайте сами…

O(h), h - высота дерева

Предшествующий и последующий элементы

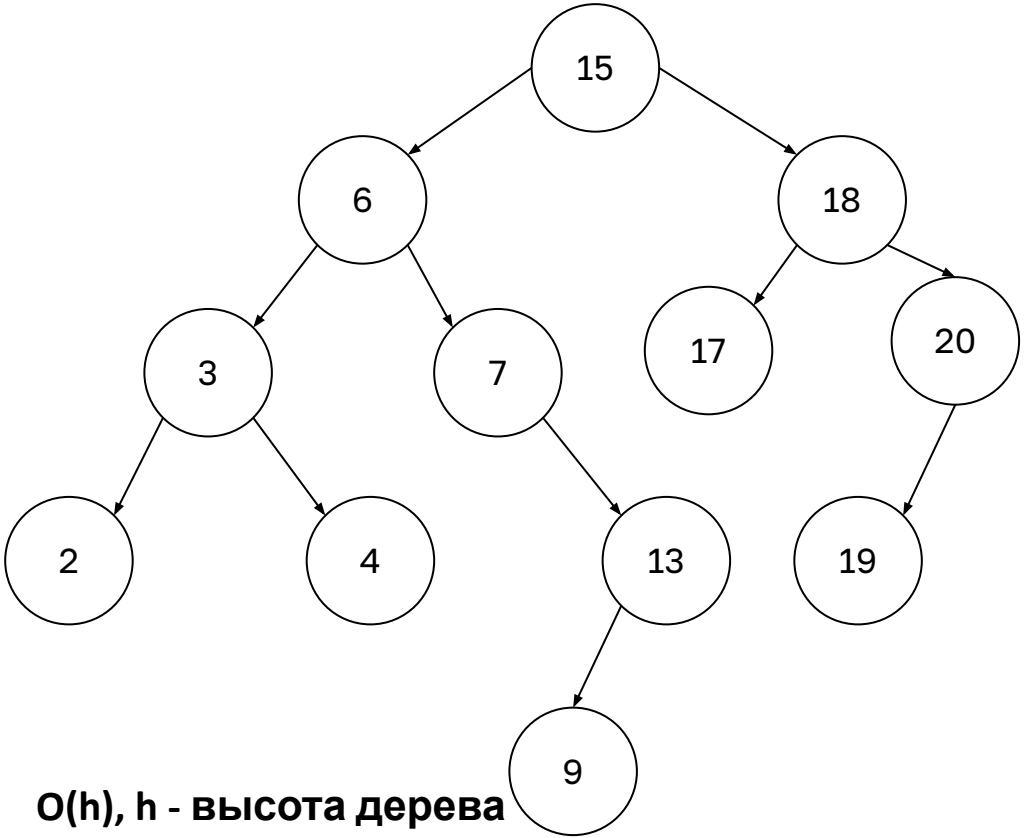




O(h), h - высота дерева

Вставка нового узла в бинарное дерево

ИТМО



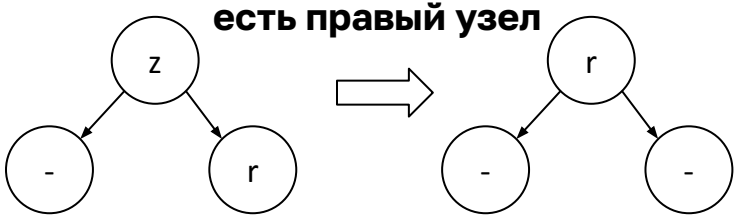
O(h), h - высота дерева

```
void insert(Tree* tree, Node* z) {
    Node* parent = nullptr;
    Node* current = tree->root;
    while (current != nullptr) {
        parent = current;
        current = z->key < current->key ?
                current->left: current->right;
    }
    z->parent = parent;

    if (parent == nullptr)
        tree->root = z;
    else if (z->key < parent->key)
        parent->left = z;
    else
        parent->right = z;
}
```

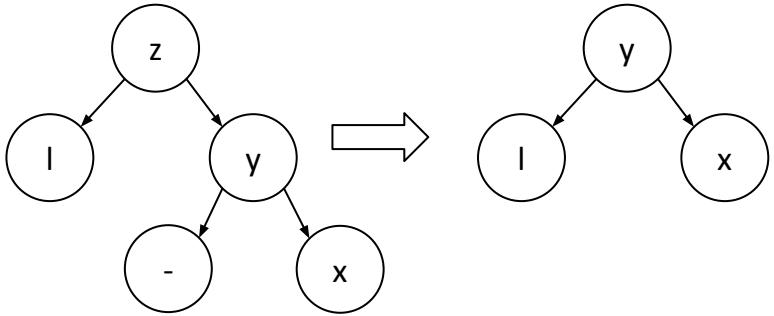
Удаление узла Z из бинарного дерева

ИТМО

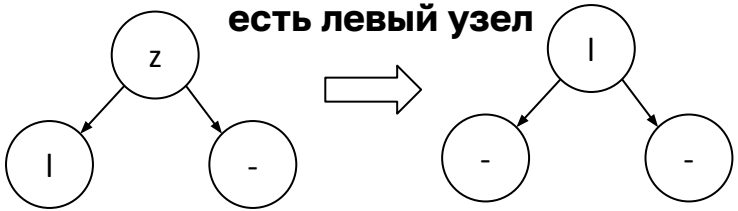
есть правый узел



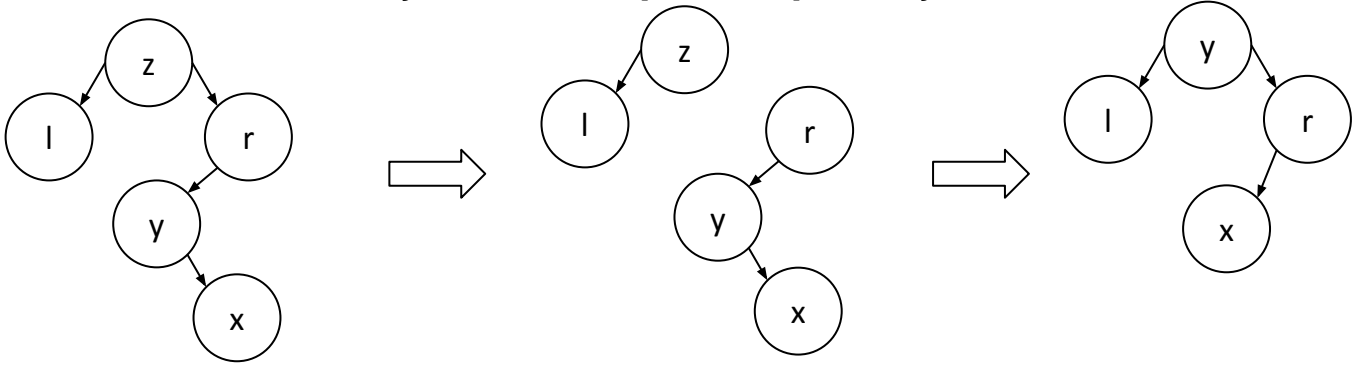
есть оба узла



есть левый узел



2 узла и поддерево в правом узле

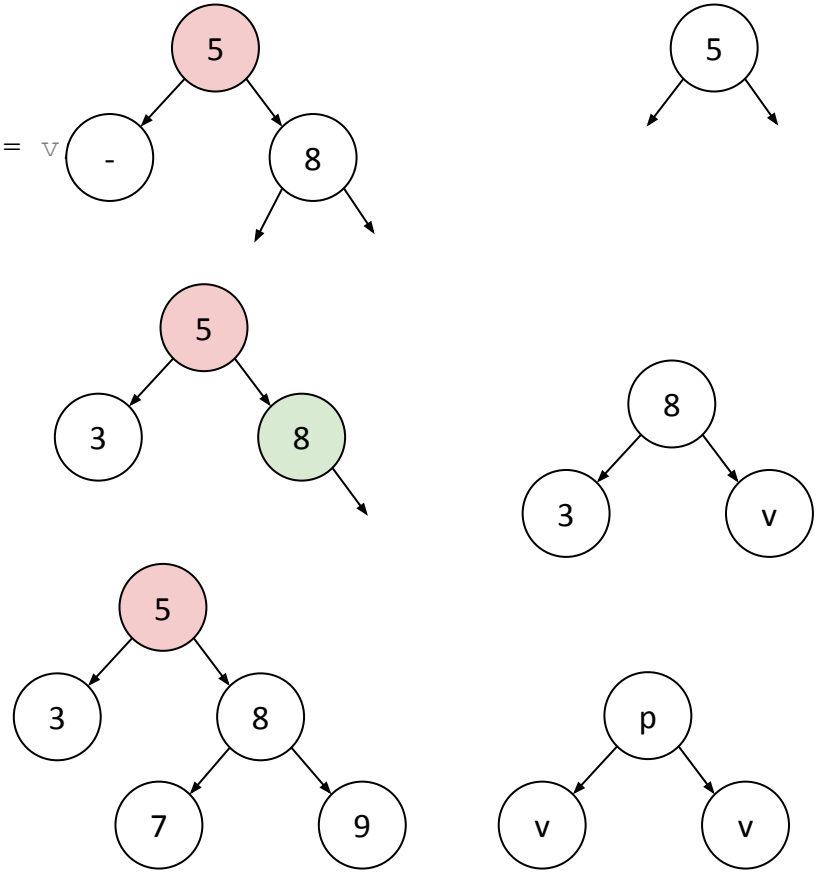


Удаление узла Z из бинарного дерева

ИТМО

```
void transplant(Tree* tree, Node* u, Node* v) {
    if (u->parent == nullptr) tree->root = v;
    else if (u == u->parent->left) u->parent->left = v
    else u->parent->right = v;
    if (v != nullptr)
        v->parent = u->parent;
}
```

```
TREE-DELETE(T, z)
if z.left == NIL
    TRANSPLANT(T, z, z.right)
elseif z.right == NIL
    TRANSPLANT(T, z, z.left)
else y = TREE-MINIMUM(z.right)
    if y.p ≠ z
        TRANSPLANT(T, y, y.right)
        y.right = z.right
        y.right.p = y
    TRANSPLANT(T, z, y)
    y.left = z.left
    y.left.p = y
```



Случайное построение бинарных деревьев поиска (Теорема 12.4)

ИТМО

Математическое ожидание высоты случайно построенного бинарного дерева поиска с n различными ключами равно O(lg n).

Определения

Высота бинарного дерева с n ключами - Xn

Экспоненциальная высота - $Yn = 2^{Xn}$

Rn - случайная величина, равная **рангу** корневого ключа, в множестве из всех ключей n (т.е. Rn содержит позицию, которую бы занимал ключ, если бы множество было бы отсортировано)

Если Rn = i -> левое поддерево корня представляет собой случайное построенное бин дерево поиска с i - 1 ключами, а правое с n - i ключами

Так как высота бин дерева на 1 больше, чем наибольшая из высот поддеревьев корневого узла, то экспоненциальная высота в 2 раза больше

если Rn = i, то
$Yn = 2 * \max(Y_{i-1}, Y_{n-i})$

n	Xn	Yn
1	1	2
3	2	4
7	3	8

Случайное построение бинарных деревьев поиска (Теорема 12.4)

ИТМО

Высота бинарного дерева с n ключами - Xn

Экспоненциальная высота - $Yn = 2^{Xn}$

Rn - случайная величина, равная **рангу** корневого ключа

Если Rn = i -> левое поддерево корня представляет собой случайное построение бин дерева поиска с i - 1 ключами, а правое с n - i ключами

Так как высота бинарного дерева на 1 больше, чем наибольшая из высот поддеревьев корневого узла, то Экспоненциальная высота больше в 2 раза

если Rn = i, то
$Yn = 2 * \max(Y_{i-1}, Y_{n-i})$

Базовый случай

Y1 = 1, так как $2^0 = 1$
Для удобства определим Y0 = 0

Далее определим индикаторные случайные величины $Z_{n,1}, Z_{n,2}, Z_{n,n}$, где
$Z_{n,i} = I\{Rn=i\}$

Поскольку Rn может быть любым из {1, 2, …, n} равновероятно, Pr{Rn == i} = 1/n
следовательно, согласно Лемме имеем:

$$E[Zn,i] = 1/n$$

поскольку ровно одно значение Zn,i равно 1, а все прочие равны 0

$$Y_n = \sum_{i=1}^{n} Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)$$

Случайное построение бинарных деревьев поиска (Теорема 12.4)

ИТМО

Высота бинарного дерева с n ключами - Xn

Экспоненциальная высота - $Y_n = 2^{X_n}$

Rn - случайная величина, равная **рангу** корневого ключа

$$Y_n = \sum_{i=1}^{n} Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)$$

Покажем, что E[Yn] полиномиально зависит от n, что неизбежно приводит к выводу, что E[Xn] = O(lg n)

Утверждаем, что индикаторная случайная переменная Zn,i = I[Rn == i] не зависит от значений Y_{i-1} и Y_{n-i}

Так как выбор Rn никак не влияет на структуру левого и правого поддеревьев, только на количество узлов в этих деревьев

Случайное построение бинарных деревьев поиска (Теорема 12.4)

ИТМО

Высота бинарного дерева с n ключами - Xn

Экспоненциальная высота - Yn = 2^{Xn}

Rn - случайная величина, равная **рангу** корневого ключа

$$Y_n = \sum_{i=1}^{n} Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)$$

Покажем, что E[Yn] полиномиально зависит от n, что неизбежно приводит к выводу, что E[Xn] = O(lg n)

$$\begin{aligned}
\mathrm{E}[Y_n] &= \mathrm{E}\left[\sum_{i=1}^{n} Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)\right] \\
&= \sum_{i=1}^{n} \mathrm{E}\left[Z_{n,i} \left(2 \cdot \max(Y_{i-1}, Y_{n-i})\right)\right] && \text{из линейности мат ожидания} \\
&= \sum_{i=1}^{n} \mathrm{E}\left[Z_{n,i}\right] \mathrm{E}\left[2 \cdot \max(Y_{i-1}, Y_{n-i})\right] && \text{из независимости} \\
&= \sum_{i=1}^{n} \frac{1}{n} \cdot \mathrm{E}\left[2 \cdot \max(Y_{i-1}, Y_{n-i})\right] && \text{из } E[Zn,i] = 1/n \\
&= \frac{2}{n} \sum_{i=1}^{n} \mathrm{E}\left[\max(Y_{i-1}, Y_{n-i})\right] && \text{(вынесли константу)} \\
&\leq \frac{2}{n} \sum_{i=1}^{n} \left(\mathrm{E}\left[Y_{i-1}\right] + \mathrm{E}\left[Y_{n-i}\right]\right)
\end{aligned}$$

Случайное построение бинарных деревьев поиска (Теорема 12.4)

ИТМО

Высота бинарного дерева с n ключами - Xn

Экспоненциальная высота - Yn = 2^{Xn}

Rn - случайная величина, равная **рангу** корневого ключа

$$\mathrm{E}[Y_n] \le \frac{2}{n}\sum_{i=1}^{n}(\mathrm{E}[Y_{i-1}] + \mathrm{E}[Y_{n-i}])$$

Поскольку, каждый член $E[Y_0]$, $E[Y_1]$, …, $E[Y_{n-1}]$ появляется в последней сумме дважды получаем следующее рекуррентное соотношение

$$\mathrm{E}[Y_n] \le \frac{4}{n}\sum_{i=0}^{n-1}\mathrm{E}[Y_i]$$

$$\begin{aligned}
\mathrm{E}[Y_n] &\le \frac{4}{n}\sum_{i=0}^{n-1}\mathrm{E}[Y_i] \\
&\le \frac{4}{n}\sum_{i=0}^{n-1}\frac{1}{4}\binom{i+3}{3} \\
&= \frac{1}{n}\sum_{i=0}^{n-1}\binom{i+3}{3} \\
&= \frac{1}{n}\binom{n+3}{4} \\
&= \frac{1}{n}\cdot\frac{(n+3)!}{4!\,(n-1)!} \\
&= \frac{1}{4}\cdot\frac{(n+3)!}{3!\,n!} \\
&= \frac{1}{4}\binom{n+3}{3}.
\end{aligned}$$

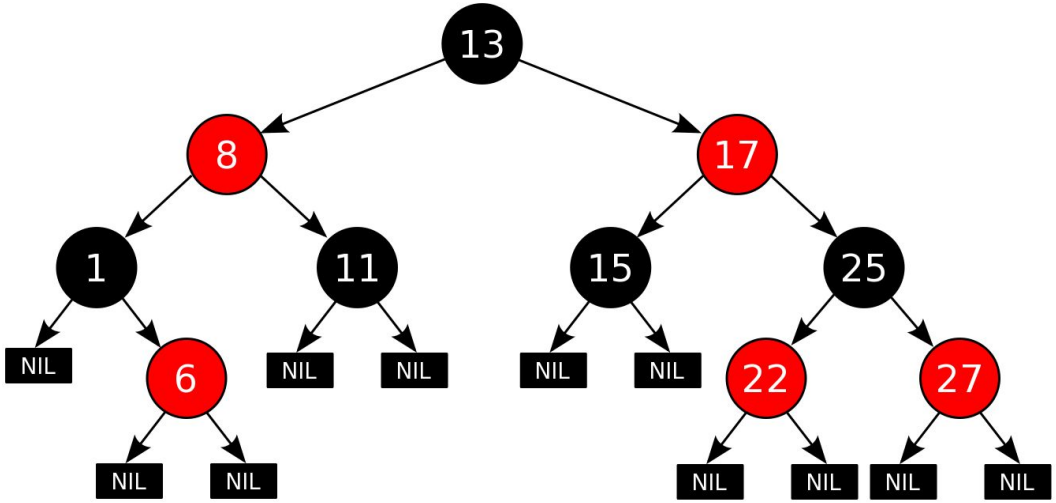
$$\begin{aligned}
2^{\mathrm{E}[X_n]} &\le \mathrm{E}\left[2^{X_n}\right] \\
&= \mathrm{E}[Y_n],
\end{aligned}$$

$$\begin{aligned}
2^{\mathrm{E}[X_n]} &\le \frac{1}{4}\binom{n+3}{3} \\
&= \frac{1}{4}\cdot\frac{(n+3)(n+2)(n+1)}{6} \\
&= \frac{n^3+6n^2+11n+6}{24}.
\end{aligned}$$

$$\mathrm{E}[X_n] = O(\lg n).$$

Красно-черные деревья

ИТМО

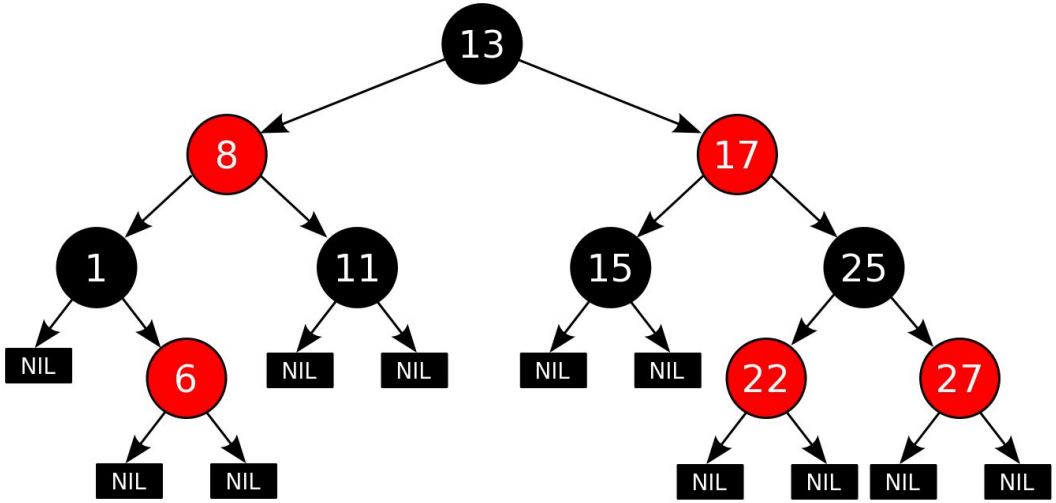


Приближенно сбалансированное дерево

Свойства:

1. Каждый узел является либо красным, либо черным.
2. **Корень дерева** является **черным** узлом
3. Каждый лист дерева **(NIL) является черным** узлом
4. Если узел **красный**, то оба его дочерних узла **черные**
5. Для каждого узла все простые пути от него до листьев, являющихся потомками данного узла, содержат **одно и то же количество черных узлов**

Красно-черные деревья




```
struct RBNode {
    int key;
    char color;
    Node* parent;
    Node* left;
    Node* right;
};
```

Лемма 13.1

Красно-черное дерево с n внутренними узлами имеет высоту, не превышающую 2lg(n + 1).

<u>Начнем с утверждения</u>: поддерево любого узла **x** содержит как минимум $2^{bh(x)} - 1$ внутренних узлов

bh(x) - количество черных узлов на любом простом пути от узла x (не считая сам узел) к листу будем называть черной высотой

если высота x == 0 -> x - лист,
поддерево содержит $2^{bh(x)} - 1$ узлов
$2^0 - 1 = 1 - 1 = 0$

иначе каждый дочерний узел имеет высоту bh(x) - 1 или bh(x) (красный или черный)

поскольку высота потомка меньше высоты x
по индукции потомки x имеют как минимум $2^{bh(x)-1} - 1$ узлов

Таким образом, с корнем x
$(2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 =$
$= 2^{bh(x)} - 1$

Лемма 13.1 (продолжение)

4. Если узел красный, то оба его дочерних узла черные

Красно-черное дерево с n внутренними узлами имеет высоту, не превышающую 2lg(n + 1).

Пусть h - высота дерева

Согласно свойству 4, по крайней мере половина узлов на пути должны быть черные

Следовательно высота должна быть как минимум h/2

$n >= 2^{h/2} - 1$

$n >= 2^{h/2} - 1$

$n + 1 >= 2^{h/2}$

$\log(n + 1) >= h / 2$

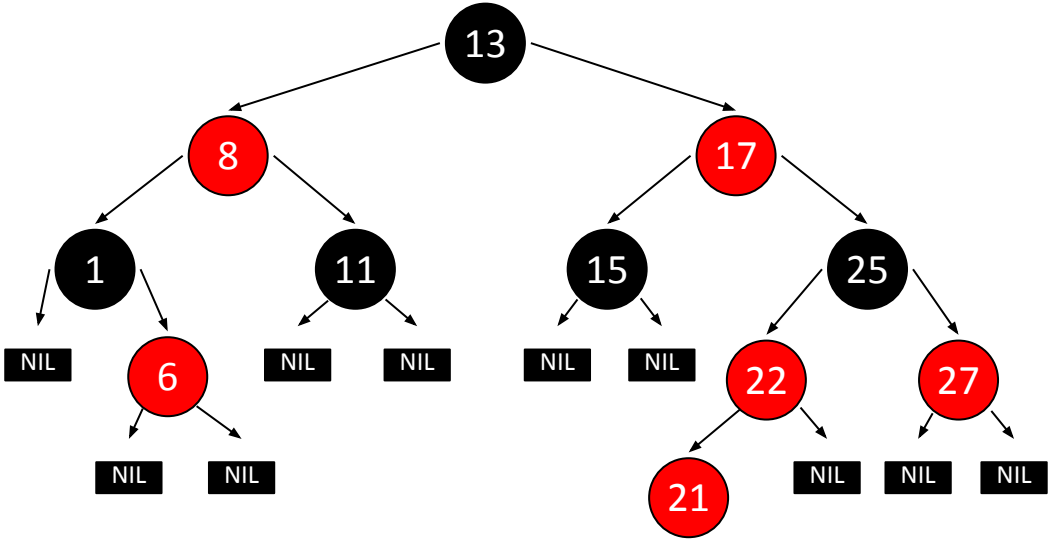
$h <= 2\lg(n + 1)$

Следовательно, операции:

- поиск
- min и max
- потомок и наследник

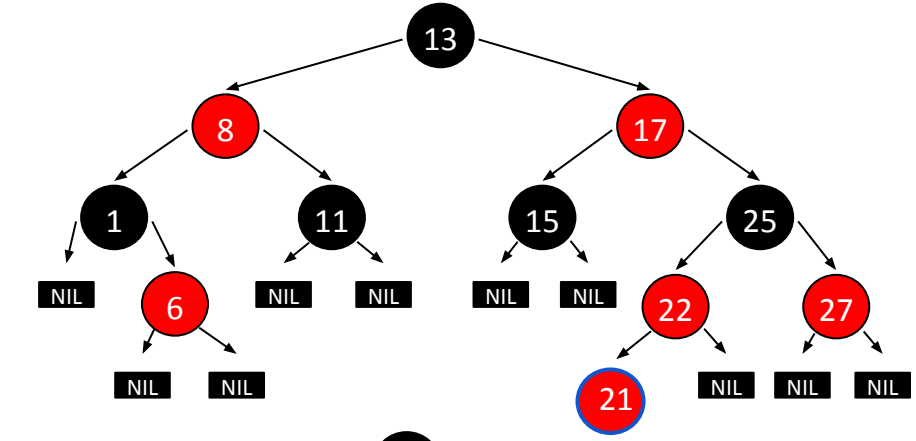
выполняются за **O(lg(N))**

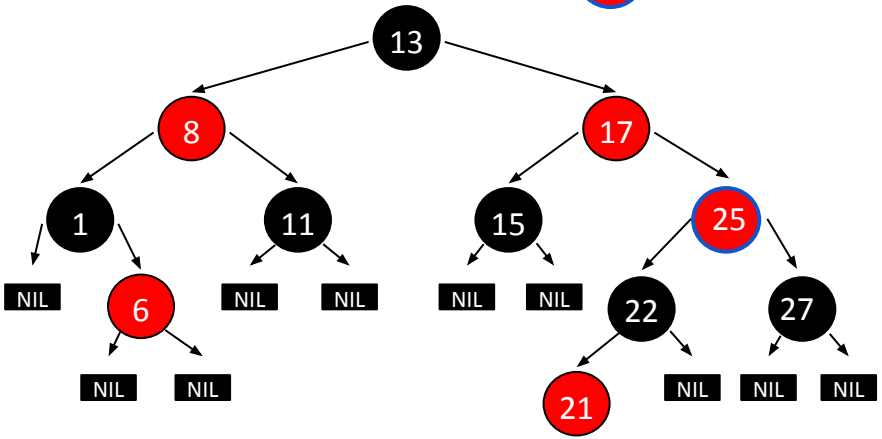
Вставка в красно-черном дереве




```
RB-INSERT(T, z)
y = T.nil
x = T.root
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y
if y == T.nil
    T.root = z
elseif z.key < y.key
    y.left = z
else y.right = z
z.left = T.nil
z.right = T.nil
z.color = RED
RB-INSERT-FIXUP(T, z)
```

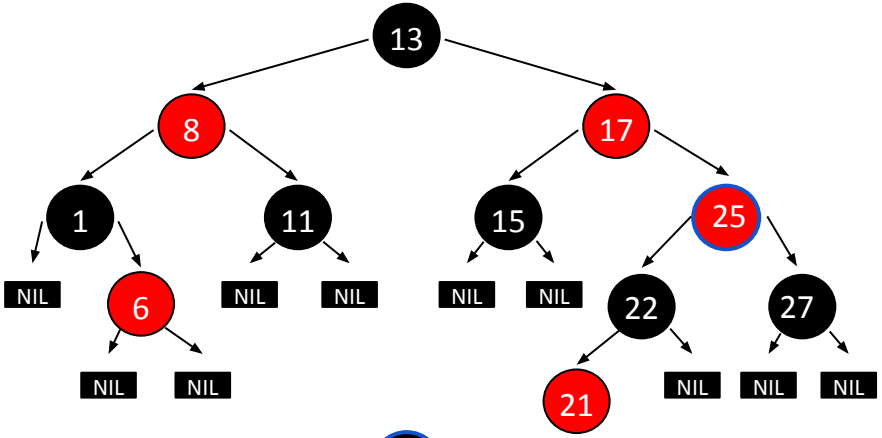
Вставка в красно-черном дереве

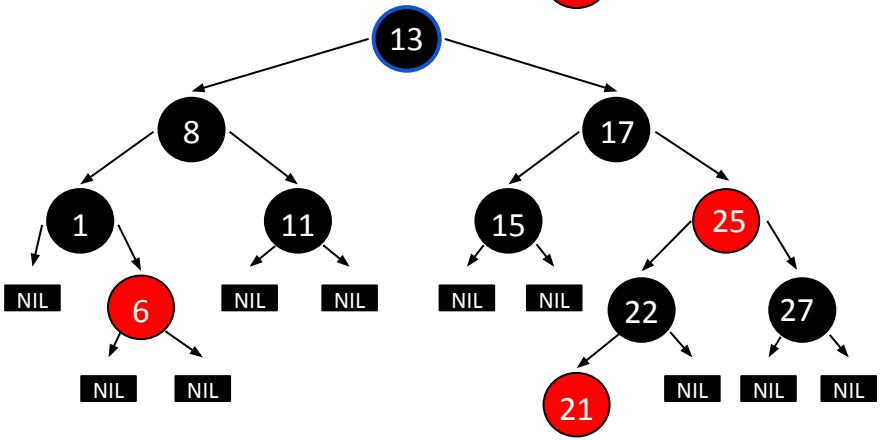





```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p
                LEFT-ROTATE(T, z)
            z.p.color = BLACK
            z.p.p.color = RED
            RIGHT-ROTATE(T, z.p.p)
    else (то же, что и в части then, но с заменой
            "правого" (right) "левым" (left) и наоборот)
T.root.color = BLACK
```

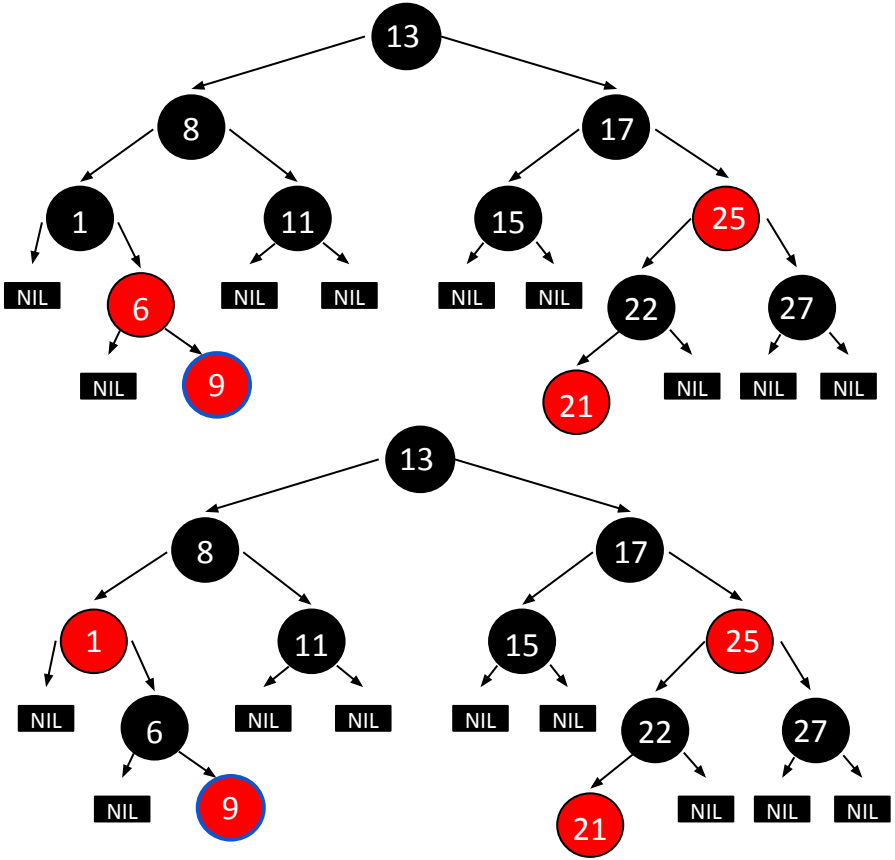
Вставка в красно-черном дереве





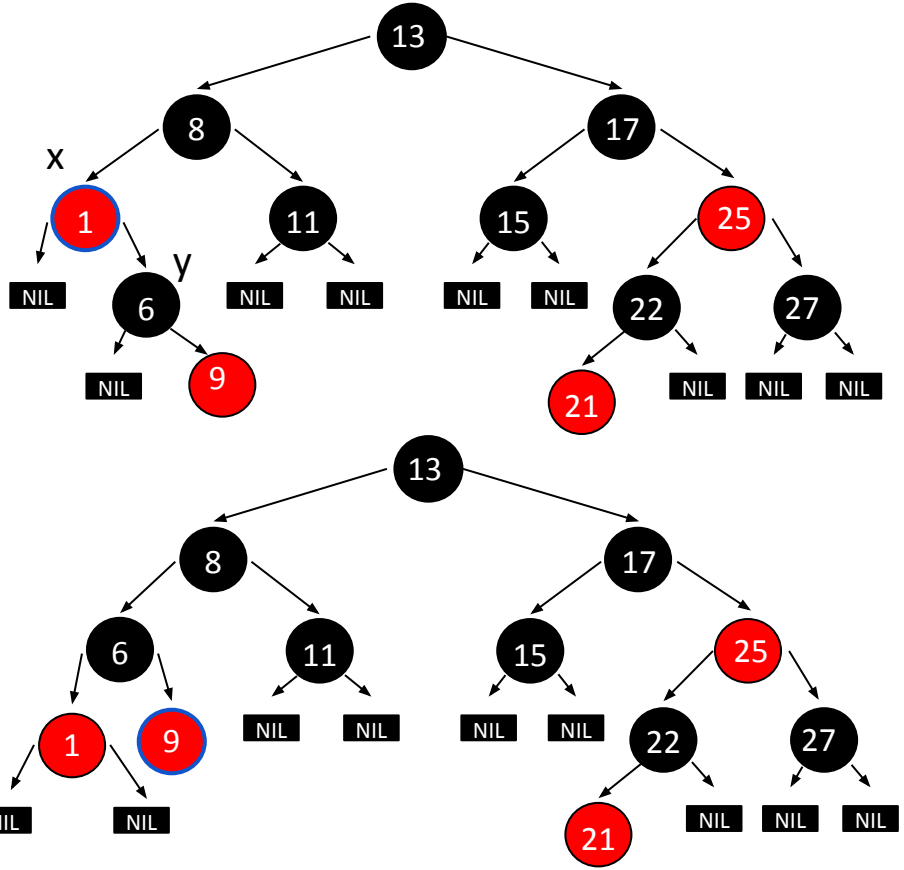

```
if z.parent == z.parent.parent.left:  # z.parent is a left child
    y = z.parent.parent.right  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.right:  # Case 2
            z = z.parent
            LEFT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        RIGHT_ROTATE(T, z.parent.parent)
else:  # z.parent is a right child
    y = z.parent.parent.left  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.left:  # Case 2
            z = z.parent
            RIGHT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        LEFT_ROTATE(T, z.parent.parent)
```

Вставка в красно-черном дереве (вставим 9)



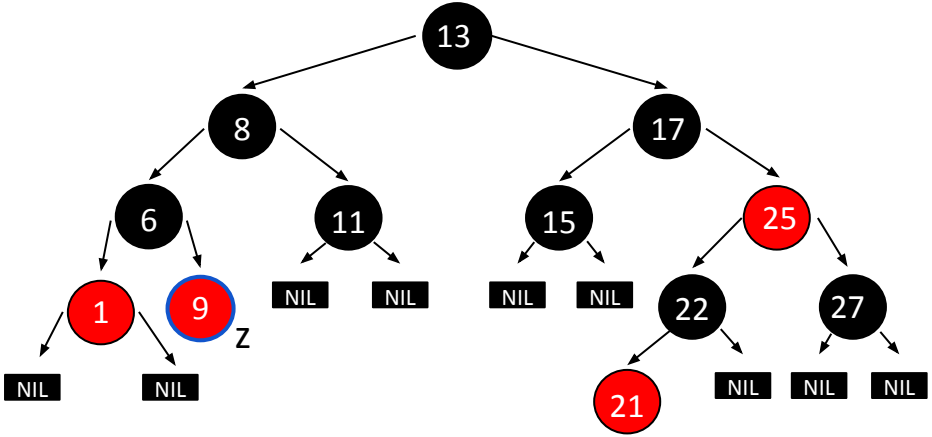

```
if z.parent == z.parent.parent.left:  # z.parent is a left child
    y = z.parent.parent.right  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.right:  # Case 2
            z = z.parent
            LEFT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        RIGHT_ROTATE(T, z.parent.parent)
else:  # z.parent is a right child
    y = z.parent.parent.left  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.left:  # Case 2
            z = z.parent
            RIGHT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        LEFT_ROTATE(T, z.parent.parent)
```

Вставка в красно-черном дереве (поворот)



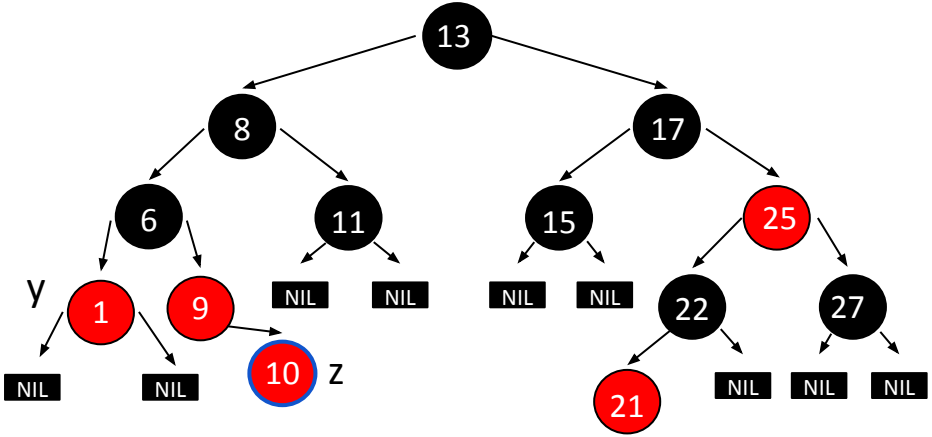

```python
def left_rotate(T, x):
    # y - правый дочерний узел x
    y = x.right
    x.right = y.left

    if y.left is not None:  # Если у y есть левый потомок
        y.left.parent = x

    y.parent = x.parent  # Переносим y к родителю x

    if x.parent is None:  # Если x был корнем
        T.root = y
    elif x == x.parent.left:  # Если x - левый потомок
        x.parent.left = y
    else:  # Если x - правый потомок
        x.parent.right = y

    y.left = x  # Перемещаем x в левое поддерево y
    x.parent = y  # Устанавливаем родителем y для x
```

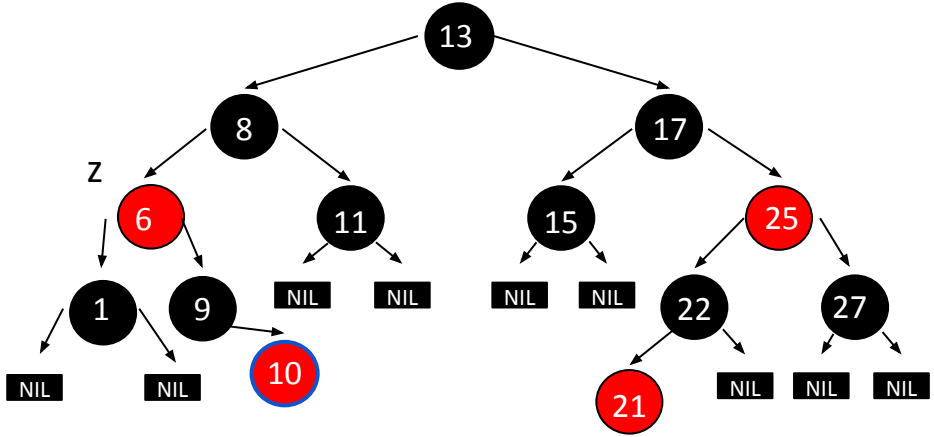
Вставка в красно-черном дереве




```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p
                LEFT-ROTATE(T, z)
            z.p.color = BLACK
            z.p.p.color = RED
            RIGHT-ROTATE(T, z.p.p)
    else (то же, что и в части then, но с заменой
            "правого" (right) "левым" (left) и наоборот)
T.root.color = BLACK
```

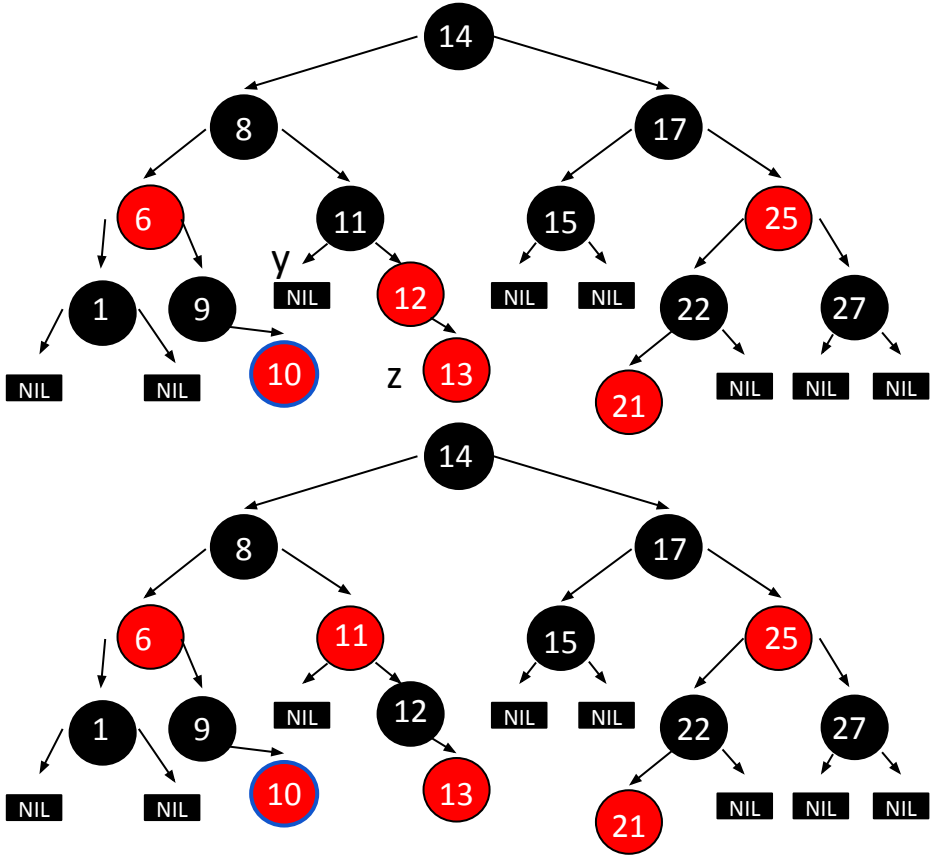
Вставка в красно-черном дереве (вставим 10)





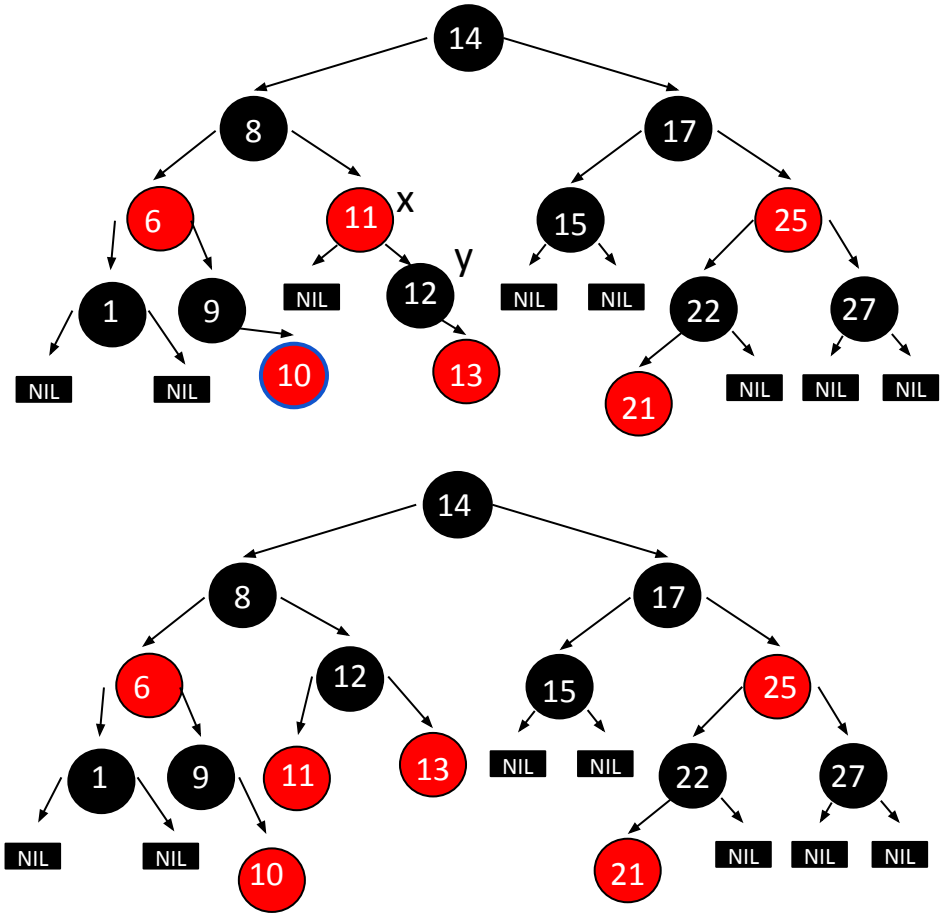

```
if z.parent == z.parent.parent.left:  # z.parent is a left child
    y = z.parent.parent.right  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.right:  # Case 2
            z = z.parent
            LEFT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        RIGHT_ROTATE(T, z.parent.parent)
else:  # z.parent is a right child
    y = z.parent.parent.left  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.left:  # Case 2
            z = z.parent
            RIGHT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        LEFT_ROTATE(T, z.parent.parent)
```

Вставка в красно-черном дереве (вставим 12 и 13)



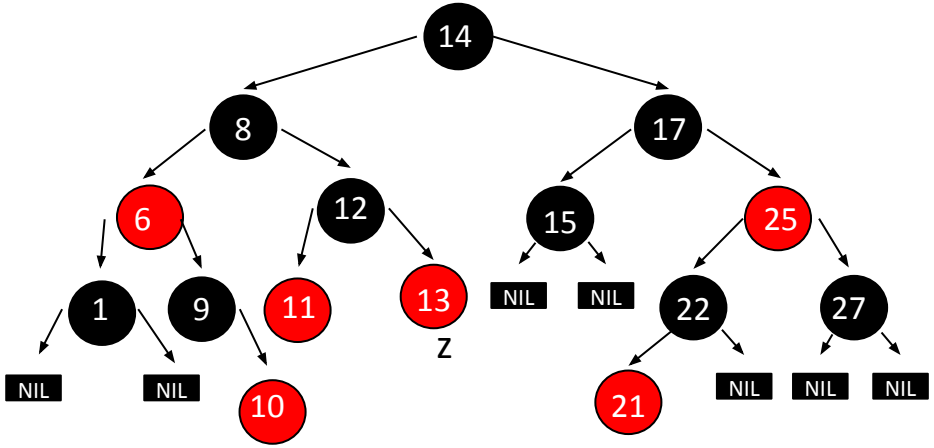

```
if z.parent == z.parent.parent.left:  # z.parent is a left child
    y = z.parent.parent.right  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.right:  # Case 2
            z = z.parent
            LEFT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        RIGHT_ROTATE(T, z.parent.parent)
else:  # z.parent is a right child
    y = z.parent.parent.left  # y is the uncle
    if y is not None and y.color == 'Red':  # Case 1
        z.parent.color = 'Black'
        y.color = 'Black'
        z.parent.parent.color = 'Red'
        z = z.parent.parent
    else:  # Case 2 and Case 3
        if z == z.parent.left:  # Case 2
            z = z.parent
            RIGHT_ROTATE(T, z)
        z.parent.color = 'Black'  # Case 3
        z.parent.parent.color = 'Red'
        LEFT_ROTATE(T, z.parent.parent)
```

Вставка в красно-черном дереве (поворот)



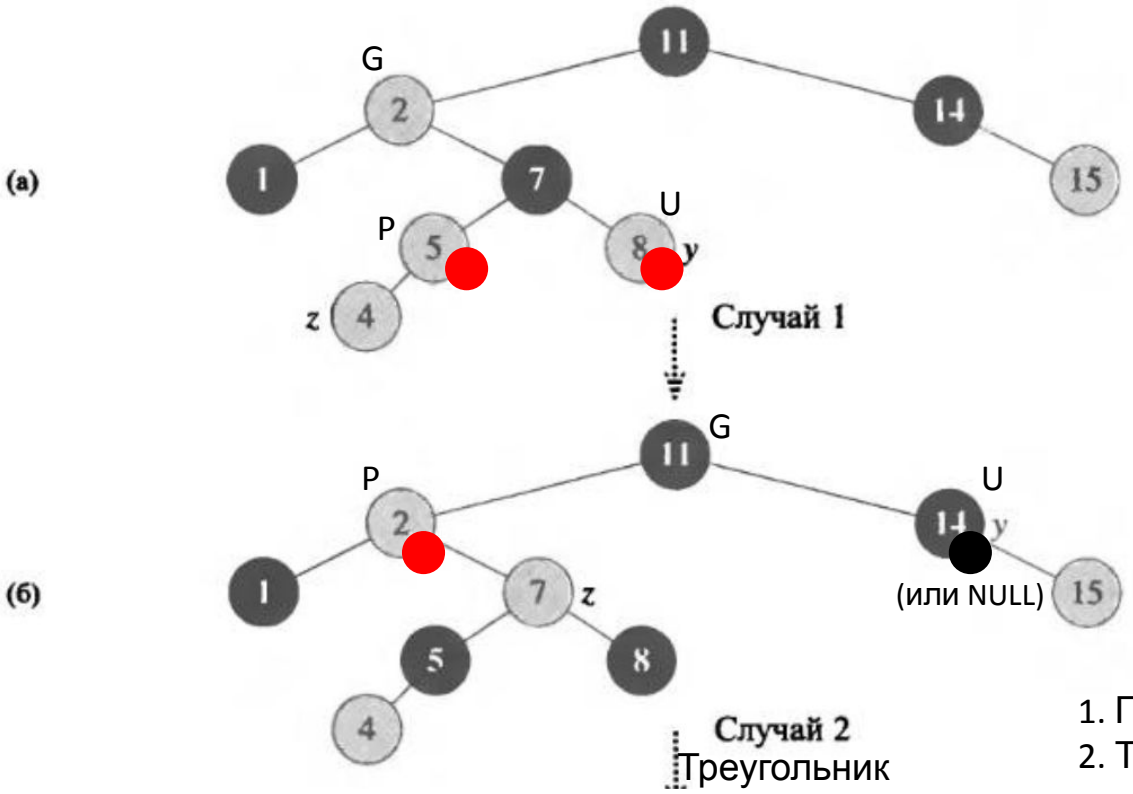

```python
def left_rotate(T, x):
    # y - правый дочерний узел x
    y = x.right
    x.right = y.left

    if y.left is not None:  # Если у у есть левый потомок
        y.left.parent = x

    y.parent = x.parent  # Переносим y к родителю x

    if x.parent is None:  # Если x был корнем
        T.root = y
    elif x == x.parent.left:  # Если x - левый потомок
        x.parent.left = y
    else:  # Если x - правый потомок
        x.parent.right = y

    y.left = x  # Перемещаем x в левое поддерево y
    x.parent = y  # Устанавливаем родителем y для x
```

Вставка в красно-черном дереве (все еще 13)




```python
def left_rotate(T, x):
    # y - правый дочерний узел x
    y = x.right
    x.right = y.left

    if y.left is not None:  # Если у у есть левый потомок
        y.left.parent = x

    y.parent = x.parent  # Переносим y к родителю x

    if x.parent is None:  # Если x был корнем
        T.root = y
    elif x == x.parent.left:  # Если x - левый потомок
        x.parent.left = y
    else:  # Если x - правый потомок
        x.parent.right = y

    y.left = x  # Перемещаем x в левое поддерево y
    x.parent = y  # Устанавливаем родителем y для x
```

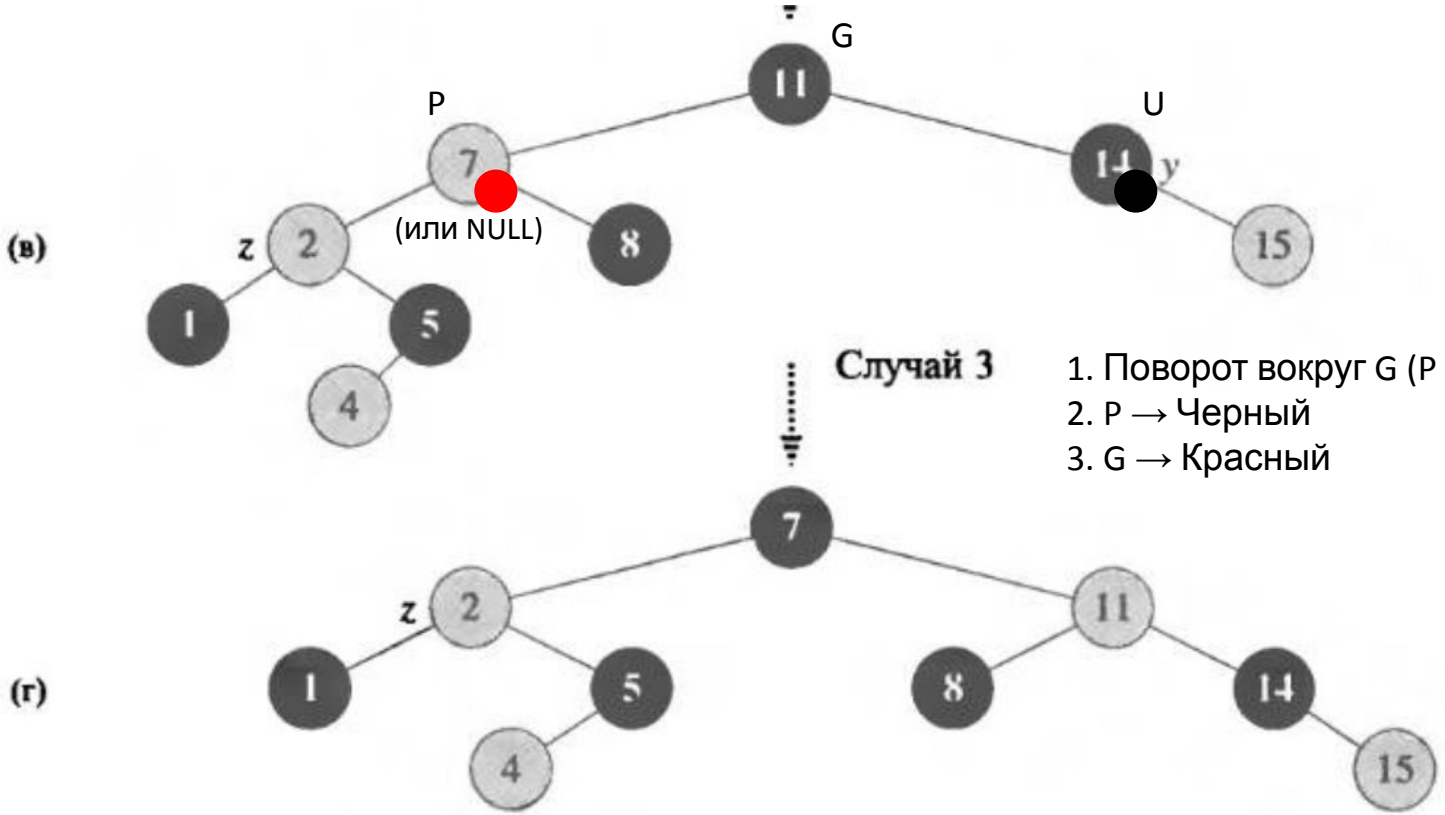
Вставка из Кормена 1



1. P и U → Черные
2. G → Красный
3. Проверять G (как новый z)

1. Поворот вокруг P (z влево)
2. Теперь это Ситуация 3 (Линия)

Вставка из Кормена 2



1. Поворот вокруг G (P вправо)
2. P → Черный
3. G → Красный

Удаление из красно-черных деревьев

ИТМО

Это как домашнее задание

Мне кажется

Или у вас есть вопросы?

ITsMOre than a UNIVERSITY

Ivan.Khodnenko@itmo.ru